Listen:

```
a = [2, "d", 5, 8, 233, "dx", 54, "we", "g", ..., 72, 23, "g"]
```

Das 4. Element

```
a[3]
```

Das Vorletzte Element

```
a[-2]
```

Das dritte bis drittletzte Element

```
a[2:-2]
```

Jedes zweite Element, beginnend ab dem vierten

```
a[3::2]
```

Jedes dritte Element in umgekehrter Reihenfolge, beginnend ab dem vorletzten

```
a[-2::-3]
```

Entfernen des siebten Elements

```
del a[6]
```

Datentypen und Ausdrücke:

```
>>> 3 + 5
8
```

normale Integer Addition

```
>>> 3 + 5.0
8.0
```

Integer Addition mit Float. Ergibt ein Float

```
>>> a = "3"+"5"
>>> a
'35'
>>> type(a)
<class 'str'>
>>>
```

String Addition hängt die Zahlen aneinander aber als String.

```
>>> b="3"*5
>>> type(b)
<class 'str'>
>>>
```

Hängt den String `"3"` fünfmal hintereinander und gibt das als String aus.

```
>>> c=3//2
>>> c
1
>>> type(c)
<class 'int'>
>>>
```

Dividiert 3 durch 2, gibt das Ergebnis als Integer aus und schneidet den Rest ab (Floor division).

```
>>> d=3/2
>>> d
1.5
>>> type(d)
<class 'float'>
>>>
```

Dividiert 3 durch 2, gibt das Ergebnis als Float aus (True division).

```
>>> int(2.71828)
2
```

Schneidet alles nach dem Komma ab (float to int).

```
>>> round(2.71828)
3
```

Rundet zur nächsten ganzen Zahl, ohne Argument.

```
>>> "hallo" + "Welt"
'halloWelt'
```

Addiert die Strings bzw. hängt die Zeichenketten aneinander.

Gerade / Ungerade

```
def is_even(number):
    assert isinstance(number, int) and number >= 1, (
        "Die gegebene Zahl ist keine positive, natürliche Zahl."
    )

    if number % 2 == 0:
        return True

    else:
        return False


assert not is_even(9)
assert is_even(10)
```

Wenn man der Funktion mit `assert is_even(-2)` keine positive natürliche Zahl, übergibt.

```
(.venv) [nordri@nordri-b9440ua CGL-vorlesung]$ python ./sheet06.py
Traceback (most recent call last):
  File "/home/nordri/Desktop/CGL-vorlesung/./sheet06.py", line 28, in <module>
    assert is_even(-2)
           ~~~~~~~^^^^
  File "/home/nordri/Desktop/CGL-vorlesung/./sheet06.py", line 18, in is_even
    assert isinstance(number, int) and number >= 1, "Die gegebene Zahl ist keine positive,\
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
AssertionError: Die gegebene Zahl ist keine positive,natürliche Zahl.
```

Man bekommt einen `AssertionError` weil die `assert`-Anweisung am Anfang (Guard Clause) die Eingabe überprüft und das Programm stoppt, bevor die Funktion vollständig ausgeführt werden kann.

Summation:

```
def get_summation_sq(number):
    assert isinstance(number, int) and number >= 1, (
        "Die gegebene Zahl ist keine positive, natürliche Zahl."
    )

    result = 0

    for k in range(1, number + 1):
        result += k * k

    return result


def get_summation_sq_test(number):
```

```
    assert isinstance(number, int) and number >= 1, (
        "Die gegebene Zahl ist keine positive, natürliche Zahl."
    )

    return (number * (number + 1) * (2 * number + 1)) / (6)

assert get_summation_sq(1) == get_summation_sq_test(1)
assert get_summation_sq(3) == get_summation_sq_test(3)
assert get_summation_sq(5) == get_summation_sq_test(5)
```

Bildet die Summe der Quadratzahlen und testet gegen die Kurzform.

Fakultät

```
def fak(n):

    assert isinstance(n, int) and n >= 0, "Die gegebene Zahl ist keine natürliche Zahl (>= 0)."

    if n == 0:
        return 1

    result = 1
    for k in range(1, n + 1):
        result *= k

    return result

assert fak(0) == 1
assert fak(1) == 1
assert fak(3) == 6
assert fak(4) == 24
```

Mit Korrektur, da $0! = 1$ definiert ist.

Eulersche Zahl approximieren

```
import math


def fak(n):
    assert isinstance(n, int) and n >= 0, (
        "Die gegebene Zahl ist keine natürliche Zahl (>= 0)."
    )

    if n == 0:
        return 1

    result = 1
    for k in range(1, n + 1):
        result *= k

    return result


assert fak(0) == 1
assert fak(1) == 1
assert fak(3) == 6
assert fak(4) == 24


def berechne_e(n_terme):
    e_approx = 0.0
    for k in range(n_terme):
        term = 1.0 / fak(k)
        e_approx += term

    return e_approx


# Genauigkeit der Approximation
N = 15
naeherung_e = berechne_e(N)

print(f"\nNäherung mit {N} Termen:")
print(f"e = {naeherung_e}")
print(f"Python's math.e = {math.e}")
```

Berechnung durch die Exponentialreihe mit $x = 1$.